

# Reflective Journal: IIoT Protocols Project

## 1. Introduction

This project focused on designing and simulating an Industrial Internet of Things (IIoT) sensor network using three different communication protocols: MQTT, CoAP, and OPC UA. The objective was to generate simulated temperature and humidity data, transmit that data using each protocol, and visualize the results in real time.

My personal goal for this project was to gain a deeper understanding of how different IIoT protocols operate in practical scenarios. Since I am pursuing a degree in artificial intelligence and working toward building skills in edge AI and distributed systems, I wanted to understand how communication between sensors and systems actually works behind the scenes. I expected the project to be technical, especially when working with asynchronous programming and configuring protocol libraries.

---

## 2. Personal Contributions

During this project, I was responsible for setting up the development environment, including creating the project directory, configuring the Python virtual environment, and installing all required libraries such as paho-mqtt, aiocoap, asyncua, pandas, and matplotlib.

I worked directly on implementing the MQTT sensor simulation script. This included connecting to the Mosquitto broker, publishing randomized temperature and humidity data every second, and formatting the payload as structured JSON. I also implemented the CoAP sensor simulation using aiocoap, where I created asynchronous functions to send POST requests containing sensor data.

For the OPC UA portion, I helped configure the server, define a namespace, create objects and variables for temperature and humidity, and continuously update those values using asynchronous loops. Additionally, I contributed to the data visualization script by subscribing to MQTT topics, decoding incoming messages, converting them into a pandas DataFrame, and plotting temperature and humidity in real time using matplotlib.

---

### **3. Learning Outcomes**

#### **MQTT**

Through MQTT, I learned how the publish-subscribe messaging model works. I understood how the broker acts as an intermediary between publishers and subscribers. Setting up and running the Mosquitto broker helped me understand how lightweight and efficient MQTT is for IoT systems that require continuous real-time data transmission. I also learned how important correct topic structure and port configuration are for successful communication.

#### **CoAP**

Working with CoAP introduced me to a request-response model that operates over UDP. I learned how asynchronous programming works in Python using `async` and `await`. At first, this was confusing because it required a different way of thinking compared to normal sequential programming. However, after implementing the simulation loop and handling responses, I gained a much better understanding of event-driven communication and non-blocking execution.

#### **OPC UA**

OPC UA felt more structured and industrial compared to MQTT and CoAP. I learned how to create an OPC UA server, define endpoints, register namespaces, create objects, and make variables writable. This protocol demonstrated how industrial systems maintain structured communication and security in enterprise environments. It showed me how large-scale automation systems organize and manage sensor data.

Overall, this project strengthened my understanding of how different communication models serve different purposes in IIoT environments.

---

### **4. Challenges and Solutions**

One of the main challenges I encountered was configuring and running the Mosquitto broker properly. Initially, I experienced connection issues when trying to publish data to localhost on port 1883. After carefully reviewing the setup steps and verifying that the broker was running before executing the simulation script, I was able to resolve the issue.

Another challenge was understanding asynchronous programming with `asyncio`. When implementing CoAP and OPC UA, I had to structure functions using `async def` and properly use `await` for tasks such as creating contexts, sending requests, and updating variables. Missing an `await` statement often caused errors. Debugging step-by-step and reviewing documentation helped me overcome these issues.

Additionally, parsing JSON payloads in the visualization script required careful handling. I had to decode the message payload, convert it into usable data, and then extract temperature and humidity values before plotting them. Ensuring the data was correctly structured prevented runtime errors during visualization.

---

## **5. Future Applications**

The knowledge gained from this project can be directly applied to future AI and IoT systems. As someone interested in edge AI and robotics systems, understanding communication protocols is essential for building distributed intelligent systems. Many real-world AI applications depend on data being transmitted efficiently from sensors to processing units.

In the future, I would like to expand this project by integrating real sensor hardware instead of simulated data. I would also compare protocol performance metrics such as latency, reliability, and scalability under different network conditions. This would provide deeper insight into which protocol is most suitable for specific industrial scenarios.

---

## **6. Conclusion**

In conclusion, the IIoT Protocols Project provided hands-on experience with MQTT, CoAP, and OPC UA in a practical simulation environment. It helped me understand how industrial sensor networks function and how different communication protocols operate under different models.

This project strengthened both my technical skills and my confidence in working with distributed systems and asynchronous programming. It also gave me a clearer understanding of how IIoT communication integrates into larger AI and automation systems.